Synthèse

Version 1.0 - 12/2023

Sommaire

1. Environnement de travail / pré-requis 3
    1.1. Installation de Git 3
    1.2. Création du compte GitHub 3
    1.3. Configuration locale de Git 3
    1.4. Configuration du compte GitHub (clé SSH) 4
2. Initialisation du projet 5
3. Préparation de la phase de développement 6
    3.1. Préparation du projet 6
    3.2. Récupération du projet 6
4. Workflow Git 7
    4.1. Définition du workflow 7
    4.2. Mise en œuvre du workflow 8
5. Références 10
6. Annexe A – Extrait d'un fichier .gitignore 11
7. Annexe B – Résolution de conflits 12
8. Annexe C – Commandes utiles 13
    8.1. Revenir à un commit antérieur 13
    8.2. Changer le message du dernier commit 13
    8.3. Renommer une branche locale 13
9. Annexe D - GitHub Quick setup 14

Tous droits réservés ©

1. Environnement de travail / pré-requis

Les sections qui suivent concernent un environnement de travail vierge de toute installation de Git et l'absence d'un compte GitHub. Si vous n'êtes pas concerné, vous pouvez aller directement à la section 2. Initialisation du projet.

1.1. Installation de Git

- Depuis le site officiel :
  - **Windows :** https://git-scm.com/download/win
    - Choisir la version *« 64-bit Git for Windows Setup »*
  - **Mac OS :** https://git-scm.com/download/mac
  - **Linux :** https://git-scm.com/download/linux
- Suivre les instructions d'installation
- Vérifier que l'installation est fonctionnelle :

```
>_ Terminal

$ git --version
git version 2.39.2
```

1.2. Création du compte GitHub

- Création d'un **compte GitHub** (https://github.com/signup)
- Renseigner le nécessaire (*e-mail* / *password* / *username*)
- Valider votre e-mail au moyen du *launch code* transmis par mail
- Répondre aux questions :
  - *How many team members will be working with you?* ***Just me***
  - *Are you a student or teacher?* ***Student***
  - *What specific features are you interested in using?* ***Collaborative coding***
- Choisir l'option ***Free*** (Continue for free)

1.3. Configuration locale de Git

Dans l'ordre :

- Définition des paramètres généraux :
  - `user.name`
  - `user.email`

Tous droits réservés ©

- Création de la clé SSH :
  - appliquer les options par défaut
  - sans *passphrase*
- Récupération de la clé SSH

Commandes correspondantes, ex. :

```
>_ Terminal
$ git config --global user.name "Nicolas Brousse"
$ git config --global user.email n.brousse@gmail.com
$ ssh-keygen -t rsa
$ cat .ssh/id_rsa.pub
```

1.4. Configuration du compte GitHub (clé SSH)

- Association de la clé SSH au compte GitHub :
  - *Account > Settings > SSH and GPG keys > New SSH key*
  - ***Title :*** nom de votre machine, ou autre
  - ***Key type :*** *Authentication Key*
  - ***Key :*** copier / coller la clé affichée via la commande `cat`

Remarques :

- La commande `cat` utilisée doit être adaptée en fonction de votre environnement de travail (Windows, Mac OS, ou Linux).
- Selon votre environnement de travail (Windows, Mac OS, ou Linux), vous pouvez être amené à suivre des instructions spécifiques (documentation officielle GitHub) : Génération d'une nouvelle clé SSH et ajout de celle-ci à ssh-agent.

Tous droits réservés ©

2. Initialisation du projet

Dans un contexte de travail collaboratif, l'initialisation du projet est réalisée par **un seul membre de l'équipe** de projet, depuis la racine du projet `<projet_git>/`. Vous trouverez un exemple de `.gitignore` en annexe.

Dans l'ordre :

1. Création du dossier de projet.
2. Création du fichier `README.md` à la racine du projet.
3. Copie du fichier `.gitignore` à la racine du projet.
4. Initialisation de Git sur le dossier du projet (*i.e.* mise en place du *versionning*).
5. Prise en compte des fichiers initiaux (`README.md` et `.gitignore`) dans le dépôt local.
6. Enregistrement via un *commit* avec ajout du commentaire associé.
7. Définition de la connexion au dépôt distant.
8. Mise à jour de la branche `main` distante.

Commandes correspondantes :

```
>_ Terminal

$ mkdir <projet_git>/
$ cd <projet_git>/
$ echo "# Titre du projet" >> README.md
$ cp <somewhere>/.gitignore .
$ git init --initial-branch=main
$ git add --all[1]
$ git commit -m "Commit initial state"
$ git remote add origin git@github.com:<id_compte_github>/<projet_git>.git
$ git push origin main[2]
```

Remarques :

- Les valeurs entre chevrons `<exemple>` sont à remplacer en fonction des spécificités de votre projet et de votre environnement.
- Par exemple, s'il est question du développement d'un blog, `<projet_git>/` doit être remplacé par quelque chose comme : `Mon-Blog/`.

[1] Ou `git -A` pour faire court.
[2] Ou `git push -u origin main` (pour paramétrer la branche amont suivie par défaut).

Tous droits réservés ©

3. Préparation de la phase de développement

3.1. Préparation du projet

La branche `main` étant en toute logique vouée à accueillir des versions stables du projet destinées à des mises en production, il est impératif de créer à ce stade la branche dédiée au développement du projet, nommée par convention `dev` ou `develop`. Cette dernière est dérivée de la branche `main` :

```
>_ Terminal

$ git checkout -b develop main
$ git push origin develop
```

Il ne reste plus qu'à vérifier l'état du projet sur GitHub.

3.2. Récupération du projet

Une fois les étapes précédentes passées, et **uniquement à partir de cet instant-là**, les autres membres de l'équipe peuvent récupérer une copie de la branche `develop` du projet, à condition d'avoir bien entendu correctement paramétré leur environnement de travail et de bénéficier des autorisations sur le dépôt Git du projet concerné :

```
>_ Terminal

$ git clone git@github.com:<id_compte_github>/<projet_git>.git -b develop
<projet_git>
$ cd <projet_git>/
```

Tous droits réservés ©

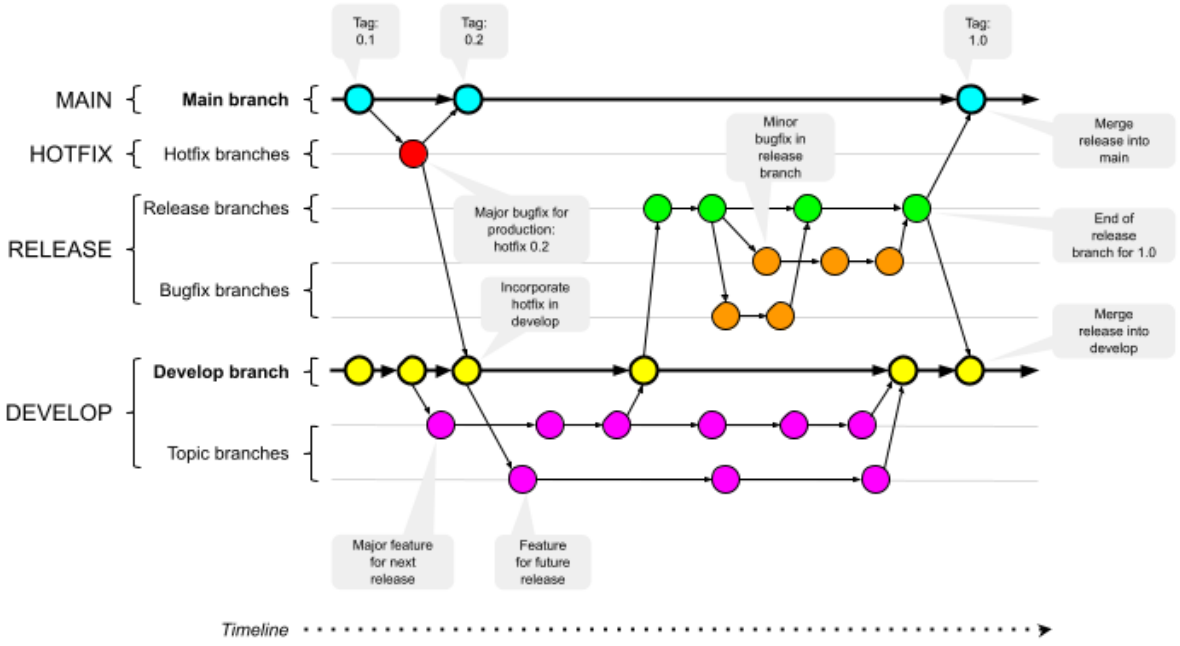
4. Workflow Git

4.1. Définition du *workflow*

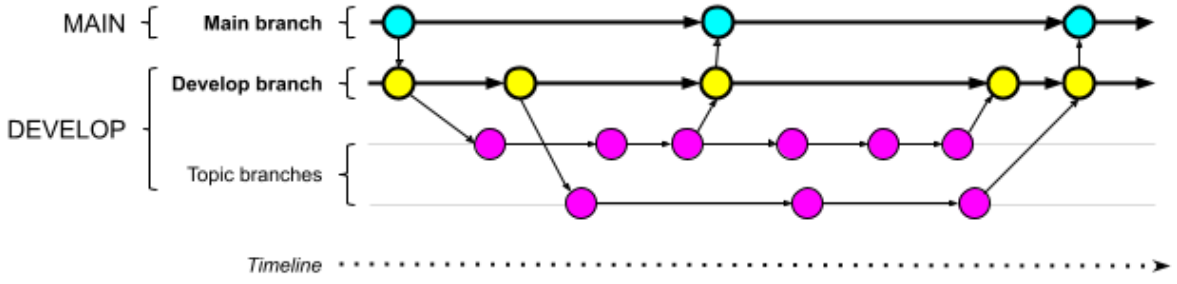
Au sein d'une équipe de projet souhaitant tirer le meilleur parti du *versionning* avec Git, il est fondamental de définir des modalités de travail, *i.e.* un ensemble de règles communes pour le *versionning* du projet. On parle ici de ***workflow***.

Il doit également exister au sein de l'équipe un membre chargé d'assurer le bon déroulement de la gestion des versions, mais aussi d'effectuer les actions majeures sur le dépôt principal du projet (ex. : fusion de la branche `develop` vers la branche `main`). À titre d'exemple, cela peut faire partie des fonctions d'un chef de projet.

Ci-dessous, un exemple de *workflow* relativement complet.



Ci-dessous, une **version simplifiée du même *workflow*** : c'est à ce modèle que se réfère la suite du document.



Tous droits réservés ©

4.2. Mise en œuvre du *workflow*

Les modalités de fonctionnement établies se matérialisent par une ***« routine »***, *i.e.* une séquence de commandes Git à répéter continuellement :

1. Accès au répertoire du projet

   ```
   cd <projet_git>
   ```

2. Vérification de l'état du projet

   ```
   git status
   ```

3. Vérification du positionnement sur le projet (sur quelle branche suis-je ?)

   ```
   git branch
   ```

4. Si nécessaire, changement de branche LOCALE (branche quelconque vers `develop`)

   ```
   git checkout develop
   ```

5. Mise à jour de la branche `develop` LOCALE à partir de la branche `develop` DISTANTE

   ```
   git pull origin develop
   ```

6. Création d'une branche LOCALE pour le travail en cours, positionnement sur cette dernière

   ```
   git checkout -b new_topic
   ```

7. Entre-temps, modification de scripts, etc. dans le projet, jusqu'à un point d'avancement suffisant

8. Vérification de l'état du projet

   ```
   git status
   ```

9. Prise en compte de toutes les modifications apportées sur la branche LOCALE de travail

   ```
   git add --all
   ```

10. Enregistrement sous forme d'un *commit* des modifications apportées, et ajout du commentaire associé

    ```
    git commit -m "Implement this and that"
    ```

11. Changement de branche LOCALE (`new_topic` vers `develop`)

    ```
    git checkout develop
    ```

Tous droits réservés ©

12. Avant fusion du travail effectué, ultime mise à jour de la branche develop LOCALE à partir de la branche develop DISTANTE (GitHub), au cas où quelqu'un aurait fait entre temps un *push* sur la branche develop DISTANTE

    `git pull origin develop`

13. Fusion de la branche de travail `new_topic` LOCALE avec la branche `develop` LOCALE[3]

    `git merge --no-ff new_topic`

14. Mise à jour de la branche develop DISTANTE

    `git push origin develop`

15. Nettoyage de l'environnement LOCAL de travail (suppression de la branche `new_topic` LOCALE)

    `git branch -d new_topic`

16. Création d'une branche LOCALE basée sur la branche develop LOCALE pour l'étape de travail suivante, positionnement sur cette dernière

    `git checkout -b next_topic develop`

17. **Retour au point n°7**, et ainsi de suite… (d'où l'idée de *« routine »*)

Résumé des commandes (*« routine »* principale) :

```
git status
git add -A
git commit -m "Implement this and that"
git checkout develop
git pull origin develop
git merge --no-ff new_topic
git push origin develop
git branch -d new_topic
git checkout -b next_topic develop
```

[3] L'option `--no-ff` engendre la création d'un *commit* de fusion, ce qui laisse une trace de la branche de travail `new_topic` dans l'historique global du projet.

Tous droits réservés ©

5. Références

- A successful Git branching model :
  - http://nvie.com/posts/a-successful-git-branching-model/
- Git Workflows and Tutorials | Atlassian Git Tutorial :
  - https://www.atlassian.com/fr/git/tutorials/comparing-workflows
- Git - Documentation (official) :
  - http://git-scm.com/documentation

Tous droits réservés ©

6. Annexe A – Extrait d'un fichier `.gitignore`

```
## ---------- Basics
# Git
/var/*
!/var/.gitignore

## ---------- Generic system files to ignore
# Windows
Thumbs.db
ehthumbs.db
Desktop.ini
$RECYCLE.BIN/
*.lnk

# Linux
*~
.directory

# OSX
.DS_Store
.AppleDouble
.LSOverride
._*
.Spotlight-V100
.Trashes

# Other
*.lock
*.swp
*.out

## ---------- JetBrains IDEs (PhpStorm, WebStorm, PyCharm, etc.)
# IntelliJ IDEA modules
*.iml

# Project settings folder
.idea/

## ---------- Etc.
```

Tous droits réservés ©

7. Annexe B – Résolution de conflits

En cours de rédaction.

Tous droits réservés ©

8. Annexe C – Commandes utiles

En cours de rédaction.

8.1. Revenir à un *commit* antérieur

En cours de rédaction.

8.2. Changer le message du dernier *commit*

En cours de rédaction.

8.3. Renommer une branche locale

En cours de rédaction.

Tous droits réservés ©

9. Annexe D - GitHub *Quick setup*

En cours de rédaction.

Tous droits réservés ©